

Module 1 Notes: Arrays, Memory, and Why Linked Lists Exist

1. Big idea of Module 1

Module 1 explains **why linked lists exist**.

Before learning nodes, pointers, insertion, deletion, and traversal, we need to understand the problem linked lists solve.

The main idea is:

Arrays are simple and fast for direct access, but their size is fixed. Linked lists are flexible in size, but slower for direct access.

So linked lists are not always better than arrays. They are useful when the program needs a structure that can grow and shrink during runtime.

2. What is an array?

An array is a collection of values stored side by side in memory.

Example:

```
int A[5];
```

This creates space for exactly 5 integers.

Memory picture:

```
A:  
[ ] [ ] [ ] [ ] [ ]  
0   1   2   3   4
```

The numbers below the boxes are indexes.

So the valid indexes are:

```
0, 1, 2, 3, 4
```

There is no index 5 because the array has only 5 spaces.

Example:

```
A[0] = 10;  
A[1] = 20;  
A[2] = 30;  
A[3] = 40;  
A[4] = 50;
```

Now the array is full.

This is wrong:

```
A[5] = 60;    // Invalid index
```

Why?

Because `A[5]` means the 6th position, but the array only has 5 positions.

3. Main limitation of arrays

Arrays have a fixed size after they are created.

That means arrays can cause two problems.

Problem 1: The array is too small

Example:

```
int A[5];
```

This array can store only 5 values.

If the user gives 6 values, the array does not have enough space.

Problem 2: The array is too large

Example:

```
int marks[1000];
```

This array can store 1000 marks.

But maybe the program only needs 30 marks.

Then 970 spaces are wasted.

So with arrays, we often have to guess the size before we know the real need.

4. Compile time vs runtime

Some decisions are made when writing or compiling the program.

Some decisions happen while the program is running.

Compile-time size decision

Example:

```
int A[5];
```

Here, the size `5` is already written in the program.

The array size is decided before the program runs.

Runtime size need

In real programs, we often do not know the required size early.

Examples:

```
How many students will register?  
How many messages will a user send?  
How many items will be added to a cart?  
How many files will be uploaded?
```

These answers may only be known while the program is running.

This is one reason linked lists are useful.

5. Stack memory

The stack is a part of memory used for local variables inside functions.

Example:

```
void example() {  
    int A[5];  
}
```

Here, `A` is a local array.

It is stored in stack memory.

Memory picture:

Stack memory:

A[0]	A[1]	A[2]	A[3]	A[4]
[]	[]	[]	[]	[]

Important point:

Array elements are stored continuously, side by side.

This is called **contiguous memory**.

6. Why array indexing is fast

Because array elements are side by side, the computer can directly calculate where an index is located.

Example:

```
A[3]
```

The computer does not need to start from `A[0]` and move one by one.

It can jump directly to index `3`.

So array access by index is:

`O(1)`

Meaning:

Direct access takes constant time.

It does not matter whether the array has 5 elements or 5000 elements. Accessing a known index is still direct.

7. Heap memory

The heap is a memory area used for dynamic allocation.

Dynamic allocation means memory is requested while the program is running.

Example:

```
int *P;  
P = new int[5];
```

Here:

- `P` is a pointer.
- `P` lives on the stack.
- The array of 5 integers lives on the heap.

Memory picture:

Stack:		Heap:
P	----->	[] [] [] [] []
		0 1 2 3 4

The pointer `P` stores the address of the heap array.

8. Heap arrays are still fixed size

A heap array is more flexible than a stack array in some ways, because we can create it during runtime.

But once created, its size is still fixed.

Example:

```
int *P = new int[5];
```

This creates an array of 5 integers.

It does not automatically become bigger if we need more space later.

To grow it, we must do extra work.

Example:

```
int *P = new int[5];

// Later, we need 10 spaces.
int *Q = new int[10];

// Copy old values from P to Q.

// Delete the old array.
delete[] P;

// Make P point to the new bigger array.
P = Q;
```

So growing an array is not simple.

We must:

1. Create a bigger array.
2. Copy old values.
3. Delete the old array.
4. Make the old pointer point to the new array.

This takes time and extra work.

9. What is a linked list?

A linked list is a data structure made of separate blocks called **nodes**.

Each node usually stores two things:

data + address of next node

Example:

[8 | next] -> [3 | next] -> [12 | NULL]

This means:

- The first node stores 8.
- The first node points to the second node.
- The second node stores 3.
- The second node points to the third node.
- The third node stores 12.
- The third node points to NULL, meaning the list ends.

A linked list does not need one large block of memory.

Instead, it uses many small nodes.

10. Linked list nodes can be scattered in memory

Array elements are side by side.

Linked list nodes do not have to be side by side.

They can be anywhere in heap memory.

Example:

Heap memory:

[8 700]	[12 NULL]	[3 900]
address 500	address 900	address 700

The actual order in memory is not important.

The list order is controlled by the stored addresses.

So the logical list is:

```
500 -> 700 -> 900 -> NULL
```

Which means:

```
8 -> 3 -> 12 -> NULL
```

The nodes are connected using pointers.

11. Meaning of NULL

`NULL` means the pointer does not point to a valid node.

In a simple linked list, the last node points to `NULL`.

Example:

```
[8 | next] -> [3 | next] -> [12 | NULL]
```

Here, `NULL` tells us:

This is the end of the list.

Without `NULL`, we would not know where the list stops.

12. Why linked lists solve the size problem

With an array, the size is decided when the array is created.

Example:

```
int A[5];
```

This array has exactly 5 spaces.

With a linked list, we can add one node at a time.

Start:

NULL

Add 8:

[8 | NULL]

Add 3:

[8 | next] -> [3 | NULL]

Add 12:

[8 | next] -> [3 | next] -> [12 | NULL]

So the list can grow as needed.

It can also shrink when nodes are removed.

This is the main reason linked lists exist.

13. Important tradeoff: arrays are faster for direct access

Arrays are good at direct access.

Example:

A[4]

If the array is:

[10]	[20]	[30]	[40]	[50]
0	1	2	3	4

Then A[4] directly gives 50.

This is fast.

But linked lists do not support direct jumping by index.

Example:

```
[10] -> [20] -> [30] -> [40] -> [50] -> NULL
```

To reach **50**, we must start from the first node and move step by step:

```
10 -> 20 -> 30 -> 40 -> 50
```

So linked list access by index is:

$O(n)$

Meaning:

In the worst case, we may need to visit many nodes.

14. Array insertion and deletion problem

Arrays are not good at inserting or deleting near the beginning.

Why?

Because elements must shift.

Example array:

```
[8] [3] [12] [20]
```

Suppose we delete **3**.

The result should be:

```
[8] [12] [20]
```

But inside the array, **12** and **20** must move left.

Dry run:

```
Before:  
[8] [3] [12] [20]  
  
Delete 3:  
[8] [12] [20] [empty]
```

This shifting costs time.

If many elements must shift, deletion can become slow.

So deleting near the beginning of an array is:

```
O(n)
```

15. Linked list deletion idea

In a linked list, we do not shift elements.

We change links.

Example:

```
[8] -> [3] -> [12] -> NULL
```

To remove `3`, we make `8` point directly to `12`.

Before:

```
[8] -> [3] -> [12] -> NULL
```

After changing the link:

```
[8] -----> [12] -> NULL  
  
[3] is removed
```

The idea is:

```
8.next = 12
```

No shifting is needed.

This is the main advantage of linked lists for insertion and deletion.

16. Complexity comparison

Operation	Array	Linked List
Access by index	$O(1)$	$O(n)$
Linear search	$O(n)$	$O(n)$
Insert/delete at beginning	$O(n)$	$O(1)$
Binary search	Good if sorted: $O(\log n)$	Inefficient
Size growth	Hard/fixed	Easy/dynamic

17. Why binary search works well on arrays

Binary search works by repeatedly checking the middle element.

Example:

```
mid = (low + high) / 2;  
A[mid];
```

In an array, `A[mid]` is fast because array access by index is $O(1)$.

So binary search works well on sorted arrays.

Its time complexity is:

```
 $O(\log n)$ 
```

18. Why binary search is inefficient on linked lists

Binary search needs quick access to the middle.

But linked lists cannot jump directly to the middle node.

To reach the middle of a linked list, we must start from the first node and move one by one.

So even if the linked list is sorted, binary search loses its main benefit.

That is why binary search is practical for sorted arrays, but not practical for linked lists.

19. When to choose an array

Choose an array when:

- You know the size in advance.
- You need fast access using indexes.
- You want to use binary search.
- You do not often insert or delete near the beginning.
- You want memory to be compact.
- You do not want extra pointer memory in every element.

Example:

Student marks for exactly 50 students

An array is suitable because the size is known.

20. When to choose a linked list

Choose a linked list when:

- The size is unknown.
- The data grows and shrinks during runtime.
- You often insert or delete near the beginning.
- You want to avoid shifting many elements.
- Records are large and expensive to move.

Example:

A music playlist where songs are added and removed often

A linked list can make sense because songs may be inserted or removed often.

21. Stack array code example

```
#include <iostream>
using namespace std;

int main() {
    int A[5] = {10, 20, 30, 40, 50};

    cout << A[0] << endl;
    cout << A[4] << endl;

    return 0;
}
```

Explanation:

- `A` is an array of 5 integers.
 - The values are stored side by side.
 - `A[0]` gives the first value.
 - `A[4]` gives the last value.
-

22. Heap array code example

```
#include <iostream>
using namespace std;

int main() {
    int *P = new int[5];

    P[0] = 10;
    P[1] = 20;
    P[2] = 30;
    P[3] = 40;
    P[4] = 50;

    cout << P[2] << endl;
}
```

```
    delete[] P;  
  
    return 0;  
}
```

Explanation:

- `P` is a pointer.
- `new int[5]` creates an array on the heap.
- `P` stores the address of that heap array.
- `delete[] P` frees the heap array.

Important:

If memory is created with `new[]`, it should be deleted with `delete[]`.

23. Single heap node example

This is a preview of linked lists.

```
#include <iostream>  
using namespace std;  
  
struct Node {  
    int data;  
    Node *next;  
};  
  
int main() {  
    Node *p = new Node;  
  
    p->data = 10;  
    p->next = nullptr;  
  
    cout << p->data << endl;  
  
    delete p;  
  
    return 0;  
}
```

Memory picture:

```
Stack:           Heap:
p -----> [ data: 10 | next: nullptr ]
```

Explanation:

- `Node` has two parts: `data` and `next`.
- `data` stores the value.
- `next` stores the address of the next node.
- `nullptr` means this node does not point to another node.
- `p` is a pointer to the node.

24. Array vs linked list: simple analogy

An array is like a fixed row of seats.

```
[ ] [ ] [ ] [ ] [ ] [ ]
```

The number of seats is fixed.

A linked list is like a treasure hunt.

```
Node 1 tells where Node 2 is.
Node 2 tells where Node 3 is.
Node 3 says the list ends.
```

Each node tells us where to go next.

25. Dry run: removing a node from a linked list

Given list:

```
8 -> 3 -> 12 -> NULL
```

Goal:

Remove `3`.

Step 1:

Find the node before 3, which is 8.

Step 2:

Make 8 point to 12.

```
8.next = 12
```

Step 3:

Remove the node containing 3.

Final list:

```
8 -> 12 -> NULL
```

Key idea:

We changed a pointer instead of shifting data.

26. Key points to remember

1. Arrays store elements side by side.
 2. Arrays have fixed size after creation.
 3. Stack arrays are local arrays stored in stack memory.
 4. Heap arrays are created dynamically, but they are still fixed size after creation.
 5. Linked lists are made of nodes.
 6. Each node stores data and a pointer to the next node.
 7. Linked list nodes can be scattered in heap memory.
 8. Linked lists can grow and shrink one node at a time.
 9. Arrays have fast index access: $O(1)$.
 10. Linked lists have slow index access: $O(n)$.
 11. Arrays need shifting for insertion/deletion near the beginning.
 12. Linked lists use pointer changes for insertion/deletion.
 13. Binary search works well on sorted arrays.
 14. Binary search is inefficient on linked lists because linked lists cannot jump to the middle.
 15. Arrays and linked lists both have advantages and disadvantages.
-

27. Common beginner mistakes

Mistake 1: Thinking heap arrays grow automatically

Wrong idea:

If an array is on the heap, it can grow by itself.

Correct idea:

A heap array is still fixed after creation.

Mistake 2: Thinking linked lists are always faster

Wrong idea:

Linked lists are better than arrays in every situation.

Correct idea:

Linked lists are better for flexible growth and some insert/delete operations, but arrays are better for direct index access.

Mistake 3: Forgetting that linked list access is sequential

Wrong idea:

I can directly access the 5th node like A[4].

Correct idea:

In a linked list, we usually start at the first node and move one node at a time.

28. Practice questions

Question 1

Why is `A[3]` fast in an array?

Answer:

Because array elements are stored side by side, so the computer can directly calculate the address of index `3`.

Question 2

Why can a linked list grow one node at a time?

Answer:

Because each new node is created separately in heap memory and connected using pointers.

Question 3

Why is binary search inefficient on a linked list?

Answer:

Because binary search needs fast access to the middle element, but a linked list must move node by node to reach the middle.

Question 4

What is the difference between a stack array and a heap array?

Answer:

A stack array is stored directly in stack memory as a local variable. A heap array is created dynamically in heap memory, and a pointer on the stack stores its address.

Question 5

How do you remove `3` from this list?

```
8 -> 3 -> 12 -> NULL
```

Answer:

Make 8 point directly to 12, then remove the node containing 3.

Final result:

```
8 -> 12 -> NULL
```

29. Module 1 final summary

Arrays and linked lists solve different problems.

Arrays are best when we know the size and need fast access by index.

Linked lists are best when the size changes often and we want to add or remove nodes without shifting many elements.

The most important tradeoff is:

Array:

Fast access, fixed size.

Linked list:

Flexible size, slower access.

That is the foundation for understanding linked lists.